

EXAMEN DE PROGRAMACIÓN – 1º GRADO SUPERIOR

Examen 19 · Patrones de diseño: Singleton, Factory, Observer, DAO y MVC

Singleton: propósito, implementación thread-safe y críticas

Factory Method y creación de objetos

Observer: sujeto y observadores desacoplados

DAO y MVC: separación de responsabilidades

Asignatura	Programación (PRG)
Curso	1º Grado Superior — Desarrollo de Aplicaciones Web (DAW)
Duración	2 horas
Puntuación total	16 puntos
Convocatoria	Evaluación 3
Valoración	Patrones de diseño

Parte 1: Opción múltiple y Verdadero/Falso

5 puntos — 0,5 cada pregunta

1. ¿Qué patrón garantiza que una clase tenga una única instancia en toda la aplicación?
- a) Singleton
 - b) Factory
 - c) Observer
 - d) Adapter

RESPUESTA / SOLUCIÓN

a) Singleton — Controla la creación para que solo exista una instancia, accesible globalmente (`getInstance()`).

-
2. ¿Qué patrón crea objetos sin que el cliente conozca la clase concreta?
- a) Factory Method
 - b) Singleton
 - c) Observer
 - d) Iterator

RESPUESTA / SOLUCIÓN

a) Factory Method — Centraliza la creación en un método que decide qué clase concreta instanciar según un parámetro o condición.

-
3. ¿Qué patrón notifica automáticamente a varios objetos cuando otro cambia de estado?

- a) Observer
- b) Strategy
- c) Proxy
- d) Decorator

RESPUESTA / SOLUCIÓN

a) Observer — El sujeto mantiene una lista de observadores y los notifica sin conocer sus clases concretas.

-
- 4. ¿Qué patrón separa la lógica de negocio, la interfaz de usuario y los datos en tres componentes?**
- a) MVC
 - b) DAO
 - c) Singleton
 - d) Facade

RESPUESTA / SOLUCIÓN

a) MVC (Modelo-Vista-Controlador) — El modelo gestiona los datos, la vista los muestra y el controlador gestiona las acciones del usuario.

-
- 5. El constructor de una clase Singleton suele ser privado para impedir que se creen instancias desde fuera.**

RESPUESTA / SOLUCIÓN

Verdadero — El constructor privado fuerza a usar getInstance(), que controla la creación de la única instancia.

6. ¿Qué patrón encapsula el acceso a la base de datos separándolo de la lógica de negocio?

- a) DAO
- b) MVC
- c) Factory
- d) Template

RESPUESTA / SOLUCIÓN

a) DAO (Data Access Object) — Aísla las operaciones de persistencia (INSERT, SELECT...) en una clase dedicada.

7. ¿Qué método expone normalmente un Singleton?

- a) getInstance()
- b) newInstance()
- c) create()
- d) instance()

RESPUESTA / SOLUCIÓN

a) getInstance() — Devuelve la instancia única, creándola la primera vez si no existe.

8. En el patrón Observer, el sujeto conoce las clases concretas de sus observadores.

RESPUESTA / SOLUCIÓN

Falso — El sujeto solo conoce la interfaz Observador (o un Consumer), lo que desacopla ambas partes.

9. En MVC, ¿qué componente gestiona las acciones del usuario (clics, teclado)?
- a) Controlador
 - b) Modelo
 - c) Vista
 - d) DAO

RESPUESTA / SOLUCIÓN

a) Controlador — Recibe las acciones del usuario, modifica el modelo y actualiza la vista.

10. ¿Qué patrón permite añadir responsabilidades a un objeto en tiempo de ejecución sin modificar su clase?
- a) Decorator
 - b) Singleton
 - c) Factory
 - d) Template Method

RESPUESTA / SOLUCIÓN

a) Decorator — Envuelve el objeto con otro que añade comportamiento (p. ej. `BufferedReader` envuelve a `FileReader`).

Parte 2: Análisis y depuración

3 puntos

1. Este Singleton no es seguro para hilos. Explica qué puede ocurrir y cómo corregirlo. (1,5 pts)

```
public class Configuracion {\n    private static Configuracion instancia;\n    public String idioma;
```

RESPUESTA / SOLUCIÓN

Problema: si dos hilos llaman a `getInstance()` a la vez y ambos ven `instancia == null`, cada uno crea su propia instancia → se rompe la unicidad.

Además el campo `idioma` es público (rompe el encapsulamiento): debería ser `private` con `getter/setter`.

Corrección (doble comprobación):

```
private static volatile Configuracion instancia;\npublic static Configuracion getInstance() {\n    if (instancia == null) {\n        synchronized (Configuracion.class) {\n            if (instancia == null) instancia = new Configuracion();\n        }\n    }\n    return instancia;\n}
```

```
}
```

O más simple: usar un enum (`enum Configuracion { INSTANCIA }`) o inicialización estática.

-
2. El siguiente método fábrica tiene un problema de mantenimiento. Identifícalo y propón una mejora. (1,5 pts)

```
public class Notificador {\n    public void enviar(String tipo, String mensaje) {\n        if (tipo.equals("email")) {\n            enviarEmail(mensaje);\n        } else if (tipo.equals("sms")) {\n            enviarSMS(mensaje);\n        } else {\n            throw new IllegalArgumentException("Tipo no válido");\n        }\n    }\n}
```

RESPUESTA / SOLUCIÓN

Problema: el método crece con cada nuevo canal (cadenas if-else) y mezcla la decisión de creación con el envío; viola el principio abierto/cerrado.

Mejora: Factory Method que devuelve una interfaz Canal con su implementación:

```
interface Canal { void enviar(String msg); }

class Email implements Canal { public void enviar(String m) { System.out.println("Email: " + m); } }

class SMS implements Canal { public void enviar(String m) { System.out.println("SMS: " + m); } }

static Canal crear(String tipo) {
    switch (tipo) {
        case "email": return new Email();
        case "sms": return new SMS();
        default: throw new IllegalArgumentException("Canal desconocido: " + tipo);
    }
}
```

Añadir un canal nuevo solo requiere una clase nueva y un case.

Parte 3: Ejercicios de programación

6 puntos

1. Implementa un Singleton Logger thread-safe (doble comprobación) con método `log(String)` que imprima con marca de tiempo, y úsalo desde dos puntos del programa. (2 ptos)

RESPUESTA / SOLUCIÓN

```
import java.time.LocalDateTime;

public class Logger {

    private static volatile Logger instancia;

    private Logger() {}

    public static Logger getInstance() {
        if (instancia == null) {
            synchronized (Logger.class) {
                if (instancia == null) instancia = new Logger();
            }
        }
        return instancia;
    }

    public void log(String msg) {
        System.out.println("[ " + LocalDateTime.now() + " ] " + msg);
    }

    public static void main(String[] args) {
        Logger l1 = Logger.getInstance();
        Logger l2 = Logger.getInstance();

        System.out.println(l1 == l2); // true: misma instancia

        l1.log("Iniciando");
        l2.log("Finalizando");
    }
}
```

2. Implementa el patrón Observer: una clase Noticias (sujeto) con `Suscribirse()/publicar()`, y una clase Suscriptor que recibe las noticias. Demuestra el funcionamiento con dos suscriptores. (2 ptos)

RESPUESTA / SOLUCIÓN

```
import java.util.ArrayList;
import java.util.List;

interface Observador { void recibir(String noticia); }

class Noticias {
    private final List<Observador> suscriptores = new ArrayList<>();
    public void suscribir(Observador o) { suscriptores.add(o); }
    public void publicar(String noticia) {
        for (Observador o : suscriptores) o.recibir(noticia);
    }
}

class Suscriptor implements Observador {
    private final String nombre;
    Suscriptor(String nombre) { this.nombre = nombre; }
    public void recibir(String noticia) {
        System.out.println(nombre + " recibió: " + noticia);
    }
}

public class DemoObserver {
    public static void main(String[] args) {
        Noticias n = new Noticias();
        n.suscribir(new Suscriptor("Ana"));
        n.suscribir(new Suscriptor("Luis"));

        n.publicar("Notas publicadas");
        // Ana recibió: Notas publicadas
        // Luis recibió: Notas publicadas
    }
}
```

3. } Diseña un DAO para una clase Alumno (id, nombre, nota): interfaz AlumnoDAO con insertar() y listar(), e implementación AlumnoDAOMemoria con una lista. Escribe también un pequeño uso de ejemplo. (2 pts)

RESPUESTA / SOLUCIÓN

```
import java.util.ArrayList;
import java.util.List;

class Alumno {
    int id; String nombre; double nota;

    Alumno(int id, String nombre, double nota) {
        this.id = id; this.nombre = nombre; this.nota = nota;
    }

    public String toString() { return id + " - " + nombre + " (" + nota + ")"; }
}

interface AlumnoDAO {
    void insertar(Alumno a);
    List<Alumno> listar();
}

class AlumnoDAOMemoria implements AlumnoDAO {
    private final List<Alumno> datos = new ArrayList<>();

    public void insertar(Alumno a) { datos.add(a); }

    public List<Alumno> listar() { return datos; }
}

public class DemoDAO {
    public static void main(String[] args) {
        AlumnoDAO dao = new AlumnoDAOMemoria();

        dao.insertar(new Alumno(1, "Ana", 8.5));
        dao.insertar(new Alumno(2, "Luis", 6.0));

        for (Alumno a : dao.listar()) System.out.println(a);
    }
}
```

Parte 4: Pregunta teórica

2 puntos

1. **Ventajas e inconvenientes de Singleton. ¿Cuándo lo usarías y cuándo lo evitarías?**

RESPUESTA / SOLUCIÓN

Ventajas: instancia única garantizada (configuración, log, pool de conexiones), acceso global cómodo, creación perezosa.

Inconvenientes: introduce estado global (dificulta los tests, que deben poder aislar/restablecer el estado), acopla el código a la clase concreta, y en entornos multihilo exige sincronización.

Usar: recursos compartidos de toda la aplicación (logger, configuración). Evitar: cuando basta inyección de dependencias o cuando la 'unicidad' no es un requisito real del dominio.

Plantilla de respuestas

(Páginas en blanco para desarrollar las soluciones)